Lab - lab - 57445 - 57445 - Google Chrome

g2.prayogshala.bits-pilani.ac.in/web-rdp/#/client/ODI1MjcAYwBwb3N0Z3Jlc3Fs?token=2AA15842E5A189DFCB148401FDEC9FC0EF73BB7F16469DE40FA2B414A32E9676

localhost:8947/edit/workspace/ML_assignment_2.ipynb?

67%

jupyter ML_assignment_2 Last Checkpoint: 48 minutes ago

File Edit View Settings Help

Trusted

Code

JupyterLab Python (conda env:base) *

[1]:
```
import pandas as pd

df = pd.read_csv("wdbc.data", header=None)

print("Dataset shape:", df.shape)
display(df.head())

features = [
    "radius", "texture", "perimeter", "area",
    "smoothness", "compactness", "concavity",
    "concave_points", "symmetry", "fractal_dimension"
]

columns = ["ID", "Diagnosis"]

for prefix in ["mean", "se", "worst"]:
    columns.extend([f"{prefix}_{feature}" for feature in features])

df.columns = columns

display(df.head())
```

Dataset shape: (569, 32)

	0	1	2	3	4	5	6	7	8	9	...	22	23	24	25	26	27	28	29
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...	25.38	17.33	184.60	2019.0	0.1622	0.6656	0.7119	0.2654
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	24.99	23.41	158.80	1956.0	0.1238	0.1866	0.2416	0.1860
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	23.57	25.53	152.50	1709.0	0.1444	0.4245	0.4504	0.2430
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	14.91	26.50	98.87	567.7	0.2098	0.8663	0.6869	0.2575
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	22.54	16.67	152.20	1575.0	0.1374	0.2050	0.4000	0.1625

5 rows × 32 columns

	ID	Diagnosis	mean_radius	mean_texture	mean_perimeter	mean_area	mean_smoothness	mean_compactness	mean_concavity	mea
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	

5 rows × 32 columns

[2]:
```
# Check missing values
print("Missing values per column:")
print(df.isnull().sum())

print("\nTotal missing values:")
print(df.isnull().sum().sum())

# Check duplicate records
print("\nDuplicate rows:")
print(df.duplicated().sum())

# Check diagnosis distribution
print("\nDiagnosis distribution:")
print(df["Diagnosis"].value_counts())
```

```
Missing values per column:
ID                       0
Diagnosis                0
                        ..

Duplicate rows:
0

Diagnosis distribution:
Diagnosis
B    357
M    212
Name: count, dtype: int64
```

[3]:
```
# Convert diagnosis to numerical target
# Malignant = 1, Benign = 0

df["target"] = df["Diagnosis"].map({
    "M": 1,
    "B": 0
})

# Check for invalid labels
if df["target"].isnull().any():
    raise ValueError("Unknown diagnosis label found")

# Remove original diagnosis column
df = df.drop(columns=["Diagnosis"])

print("Target distribution:")
print(df["target"].value_counts())

display(df.head())
```

```
Target distribution:
target
0    357
1    212
Name: count, dtype: int64
```

	ID	mean_radius	mean_texture	mean_perimeter	mean_area	mean_smoothness	mean_compactness	mean_concavity	mean_concave_
0	842302	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0
1	842517	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0
2	84300903	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0
3	84348301	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0
4	84358402	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0

5 rows × 32 columns

[4]:
```
# Remove identifier columns if present
df = df.drop(columns=["ID", "id", "errors="ignore")

# X = input features
X = df.drop(columns=["target"])

# y = output label
y = df["target"]

print("Features shape:", X.shape)
print("Target shape:", y.shape)

print("Number of features:", X.shape[1])
print("Number of samples:", X.shape[0])

# Validate assignment requirements
assert X.shape[0] >= 500, "Minimum 500 samples required"
assert X.shape[1] >= 12, "Minimum 12 features required"
```

```
Features shape: (569, 30)
Target shape: (569,)
Number of features: 30
Number of samples: 569
```

[5]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("X_train:", X_train.shape)
print("X_test:", X_test.shape)
print("y_train:", y_train.shape)
print("y_test:", y_test.shape)
```

```
X_train: (455, 30)
X_test: (114, 30)
y_train: (455,)
y_test: (114,)
```

[6]:
```
# Create training dataset
train_data = X_train.copy()
train_data["target"] = y_train

# Create testing dataset
test_data = X_test.copy()
test_data["target"] = y_test

# Save files
train_data.to_csv("train_data.csv", index=False)
test_data.to_csv("test_data.csv", index=False)

print("Training CSV:", train_data.shape)
print("Testing CSV:", test_data.shape)

print("\nTesting target distribution:")
print(test_data["target"].value_counts())
```

```
Training CSV: (455, 31)
Testing CSV: (114, 31)

Testing target distribution:
target
0    72
1    42
Name: count, dtype: int64
```

26°C Mostly cloudy

Search

ENG IN

Lab - lab - 57445 - 57445 - Google Chrome

g2.prayogshala.bits-pilani.ac.in/web-rdp/#/client/ODI1MjcAYw8wb3N0Z3Jlc3Fs?token=2AA15842E5A189DFCB148401FDEC9FC0EF738B7F16469DE40FA2B414A32E9676

localhost:8947/edit/workspace/ML_assignment_2.ipynb?

jupyter ML_assignment_2 Last Checkpoint: 52 minutes ago

File Edit View Settings Help

Trusted

JupyterLab Python [conda env:base]

```
train_df = X_train.copy()
train_df["target"] = y_train

test_df = X_test.copy()
test_df["target"] = y_test

train_df.to_csv("train_data.csv", index=False)
test_df.to_csv("test_data.csv", index=False)

print("Files saved successfully!")
```

```
Files saved successfully!
```

```
# Reload saved test dataset
test_df = pd.read_csv("test_data.csv")

print("Test dataset shape:", test_df.shape)

# Verify correct target labels
print("\nTarget distribution:")
print(test_df["target"].value_counts())

# Verify feature columns
assert test_df.drop(columns=["target"]).columns.tolist() == X.columns.tolist()

# Verify missing values
assert test_df.isnull().sum().sum() == 0

# Verify target labels
assert set(test_df["target"].unique()) == {0, 1}

print("\nPhase 1 verification successful!")
display(test_df.head())
```

```
Test dataset shape: (114, 31)

Target distribution:
target
0    72
1    42
Name: count, dtype: int64

Phase 1 verification successful!
```

	mean_radius	mean_texture	mean_perimeter	mean_area	mean_smoothness	mean_compactness	mean_concavity	mean_concave_points	me
0	11.41	10.82	73.34	403.3	0.09373	0.06685	0.03512	0.02623	
1	20.94	23.56	138.90	1364.0	0.10070	0.16060	0.27120	0.13100	
2	16.17	16.07	106.30	788.5	0.09880	0.14380	0.06651	0.05397	
3	14.42	19.77	94.48	642.5	0.09752	0.11410	0.09388	0.05839	
4	13.38	30.72	86.34	557.2	0.09245	0.07426	0.02819	0.03264	

5 rows × 31 columns

```
print("Logistic Regression training completed successfully!")
```

```
Logistic Regression training completed successfully!
```

```
# Phase 2 - Logistic Regression Prediction

# Predict diagnosis labels
lr_pred = lr_model.predict(X_test)

# Predict probability of malignant class (1)
lr_prob = lr_model.predict_proba(X_test)[:, 1]

print("First 10 actual labels:")
print(y_test.iloc[:10].to_numpy())

print("\nFirst 10 predicted labels:")
print(lr_pred[:10])

print("\nFirst 10 malignant probabilities:")
print(lr_prob[:10])


# Compare actual and predicted values

prediction_df = pd.DataFrame({
    "Actual": y_test.to_numpy(),
    "Predicted": lr_pred,
    "Malignant Probability": lr_prob
})

display(prediction_df.head(10))
```

```
First 10 actual labels:
[0 1 0 1 0 0 1 0 0 0]

First 10 predicted labels:
[0 1 0 1 1 0 1 0 0 0]

First 10 malignant probabilities:
[3.64568116e-04 9.99989989e-01 4.25679471e-02 5.76187322e-01
 5.09155489e-01 6.16861620e-04 7.61534086e-01 1.01217174e-03
 5.68725516e-04 1.07675795e-02]
```

	Actual	Predicted	Malignant Probability
0	0	0	0.000365
1	1	1	1.000000
2	0	0	0.042568
3	1	1	0.576187
4	0	1	0.509155
5	0	0	0.000616
6	1	1	0.761534
7	0	0	0.001012
8	0	0	0.000569
9	0	0	0.010768

```
# Phase 2 - Logistic Regression Evaluation

# 1. Accuracy
lr_accuracy = accuracy_score(y_test, lr_pred)

# 2. AUC (Area Under ROC Curve)
lr_auc = roc_auc_score(y_test, lr_prob)

# 3. Precision
lr_precision = precision_score(
    y_test, lr_pred, zero_division=0
)

# 4. Recall
lr_recall = recall_score(
    y_test, lr_pred, zero_division=0
)

# 5. F1 Score
lr_f1 = f1_score(
    y_test, lr_pred, zero_division=0
)

# 6. Matthews Correlation Coefficient
lr_mcc = matthews_corrcoef(y_test, lr_pred)

# Display results
print("LOGISTIC REGRESSION - EVALUATION RESULTS")
print("-" * 45)

print(f"Accuracy : {lr_accuracy:.4f}")
print(f"AUC      : {lr_auc:.4f}")
print(f"Precision: {lr_precision:.4f}")
print(f"Recall   : {lr_recall:.4f}")
print(f"F1 Score : {lr_f1:.4f}")
print(f"MCC      : {lr_mcc:.4f}")
```

```
LOGISTIC REGRESSION - EVALUATION RESULTS
---------------------------------------------
Accuracy : 0.9649
AUC      : 0.9960
Precision: 0.9750
Recall   : 0.9286
F1 Score : 0.9512
MCC      : 0.9245
```

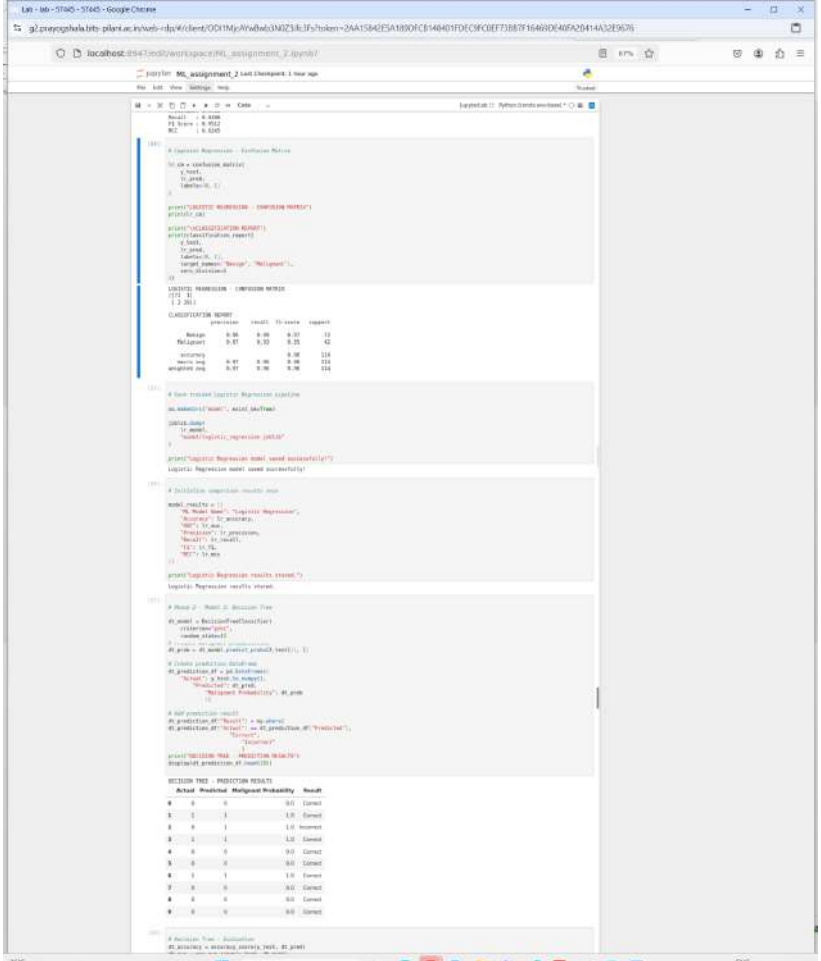
```
# Logistic Regression - Confusion Matrix

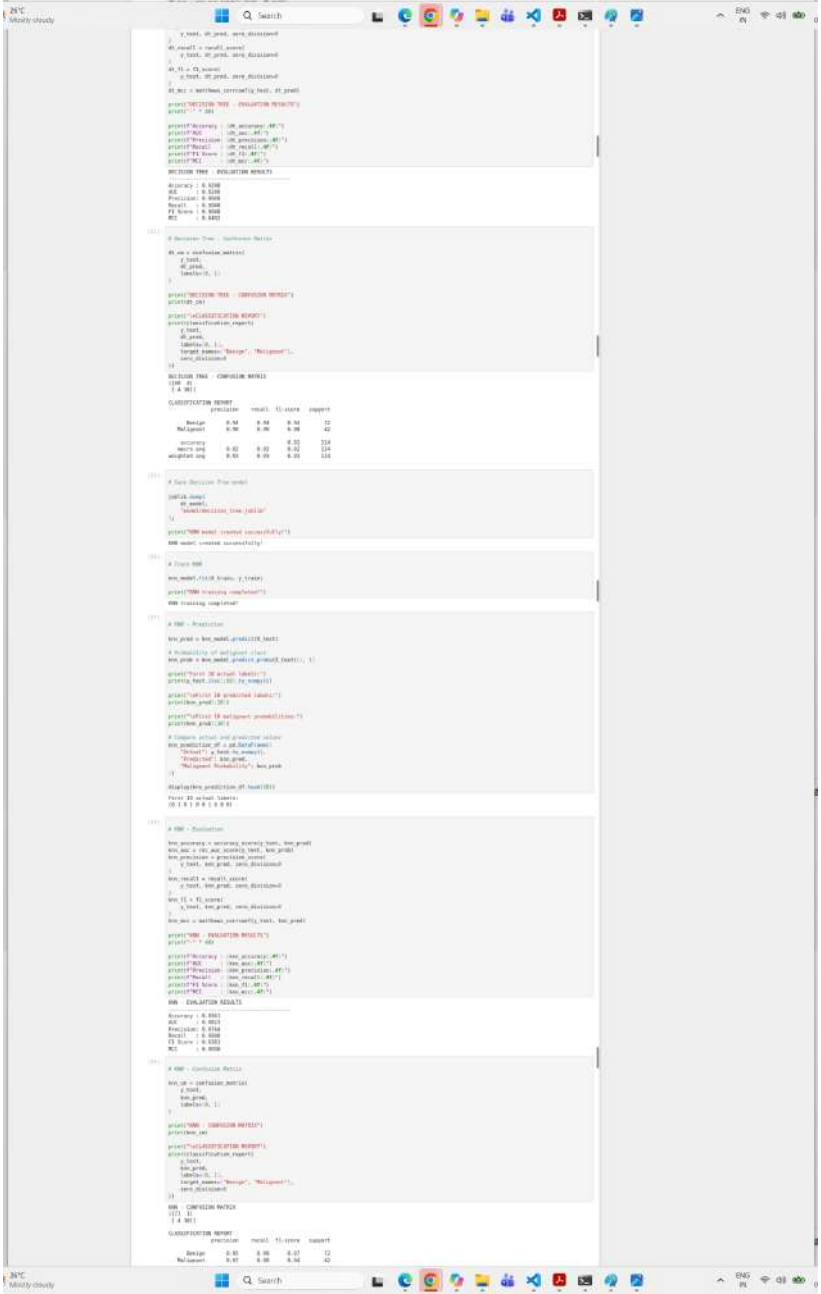
lr_cm = confusion_matrix(
    y_test,
    lr_pred
```

26°C Mostly cloudy

Search

ENG IN





Lab - lab - 57445 - 57445 - Google Chrome

g2.prayogshala.bits-pilani.ac.in/web-rdp/#/client/ODI1MjcAYwBwb3N0Z3Jlc3Fs?token=2AA15842E5A189DFCB148401FDEC9FC0EF73BB7F16469DE40FA2B414A32E9676

localhost:8947/edit/workspace/ML_assignment_2.ipynb?

jupyter ML_assignment_2 Last Checkpoint: 1 hour ago

File Edit View Settings Help

Trusted

Code

JupyterLab Python [conda env:base] *

```
print(classification_report(
    y_test,
    knn_pred,
    labels=[0, 1],
    target_names=["Benign", "Malignant"],
    zero_division=0
))
```

```
KNN - CONFUSION MATRIX
[[71  1]
 [ 4 38]]

CLASSIFICATION REPORT
              precision    recall  f1-score   support

      Benign       0.95      0.99      0.97        72
   Malignant       0.97      0.90      0.94        42

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114
```

[36]:

```
# Save KNN model

joblib.dump(
    knn_model,
    "model/knn.joblib"
)

print("KNN model saved successfully!")
```

```
KNN model saved successfully!
```

[37]:

```
# Add KNN results

model_results.append({
    "ML Model Name": "KNN",
    "Accuracy": knn_accuracy,
    "AUC": knn_auc,
    "Precision": knn_precision,
    "Recall": knn_recall,
    "F1": knn_f1,
    "MCC": knn_mcc
})

print("KNN results stored successfully!")

display(pd.DataFrame(model_results).round(4))
```

```
KNN results stored successfully!
```

	ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
0	Logistic Regression	0.9649	0.9960	0.9750	0.9286	0.9512	0.9245
1	Decision Tree	0.9298	0.9246	0.9048	0.9048	0.9048	0.8492
2	KNN	0.9561	0.9823	0.9744	0.9048	0.9383	0.9058

[32]:

```
# Phase 2 - Model 4: Gaussian Naive Bayes

nb_model = GaussianNB()

print("Gaussian Naive Bayes model created successfully!")
```

```
Gaussian Naive Bayes model created successfully!
```

[33]:

```
# Train Gaussian Naive Bayes

nb_model.fit(X_train, y_train)

print("Gaussian Naive Bayes training completed!")
```

```
Gaussian Naive Bayes training completed!
```

[34]:

```
# Gaussian Naive Bayes - Prediction

# Predict diagnosis labels
nb_pred = nb_model.predict(X_test)

# Predict probability of malignant class
nb_prob = nb_model.predict_proba(X_test)[:, 1]

print("First 10 actual labels:")
print(y_test.iloc[:10].to_numpy())

print("\nFirst 10 predicted labels:")
print(nb_pred[:10])

print("\nFirst 10 malignant probabilities:")
print(nb_prob[:10])

# Compare actual and predicted values
nb_prediction_df = pd.DataFrame({
    "Actual": y_test.to_numpy(),
    "Predicted": nb_pred,
    "Malignant Probability": nb_prob
})

display(nb_prediction_df.head(10))
```

```
First 10 actual labels:
[0 1 0 1 0 0 1 0 0 0]

First 10 predicted labels:
[0 1 0 0 0 0 0 0 0 0]

First 10 malignant probabilities:
[3.75512810e-15 1.00000000e+00 1.58085749e-02 3.20224578e-03
 2.92922538e-10 1.00665138e-02 2.64632029e-01 8.64760807e-11
 1.19101759e-15 1.14531854e-12]
```

	Actual	Predicted	Malignant Probability
0	0	0	3.755128e-15
1	1	1	1.000000e+00
2	0	0	1.580857e-02
3	1	0	3.202246e-03
4	0	0	2.929225e-10
5	0	0	1.006651e-02
6	1	0	2.646320e-01
7	0	0	8.647609e-11
8	0	0	1.191018e-15
9	0	0	1.145319e-12

[35]:

```
# Gaussian Naive Bayes - Evaluation

nb_accuracy = accuracy_score(y_test, nb_pred)
nb_auc = roc_auc_score(y_test, nb_prob)
nb_precision = precision_score(
    y_test, nb_pred, zero_division=0
)
nb_recall = recall_score(
    y_test, nb_pred, zero_division=0
)
nb_f1 = f1_score(
    y_test, nb_pred, zero_division=0
)
nb_mcc = matthews_corrcoef(y_test, nb_pred)

print("GAUSSIAN NAIVE BAYES - EVALUATION RESULTS")
print("-" * 45)

print(f"Accuracy : {nb_accuracy:.4f}")
print(f"AUC      : {nb_auc:.4f}")
print(f"Precision: {nb_precision:.4f}")
print(f"Recall   : {nb_recall:.4f}")
print(f"F1 Score : {nb_f1:.4f}")
print(f"MCC      : {nb_mcc:.4f}")
```

```
GAUSSIAN NAIVE BAYES - EVALUATION RESULTS
---------------------------------------------
Accuracy : 0.9386
AUC      : 0.9934
Precision: 1.0000
Recall   : 0.8333
F1 Score : 0.9091
MCC      : 0.8715
```

[36]:

```
# Gaussian Naive Bayes - Confusion Matrix

nb_cm = confusion_matrix(
    y_test,
    nb_pred,
    labels=[0, 1]
)

print("GAUSSIAN NAIVE BAYES - CONFUSION MATRIX")
print(nb_cm)

print("\nCLASSIFICATION REPORT")
print(classification_report(
    y_test,
    nb_pred,
    labels=[0, 1],
    target_names=["Benign", "Malignant"],
    zero_division=0
))
```

```
GAUSSIAN NAIVE BAYES - CONFUSION MATRIX
[[72  0]
 [ 7 35]]

CLASSIFICATION REPORT
              precision    recall  f1-score   support

      Benign       0.91      1.00      0.95        72
   Malignant       1.00      0.83      0.91        42

    accuracy                           0.94       114
   macro avg       0.96      0.92      0.93       114
weighted avg       0.94      0.94      0.94       114
```

[37]:

```
# Save Gaussian Naive Bayes model

joblib.dump(
    nb_model,
    "model/naive_bayes.joblib"
)

print("Gaussian Naive Bayes model saved successfully!")
```

```
Gaussian Naive Bayes model saved successfully!
```

26°C Mostly cloudy

Search

ENG IN

Lab - lab - 57445 - 57445 - Google Chrome

g2.prayogshala.bits-pilani.ac.in/web-rdp/#/client/ODI1MjcAYwBwb3N0Z3Jlc3Fs?token=2AA15842E5A189DFCB148401FDEC9FC0EF73BB7F16469DE40FA2B414A32E9676

localhost:8947/edit/workspace/ML_assignment_2.ipynb?

jupyter ML_assignment_2 Last Checkpoint: 1 hour ago

File Edit View Settings Help

Trusted

JupyterLab Python [conda env:base] *

```
           accuracy                           0.94       114
          macro avg       0.96      0.92      0.93       114
       weighted avg       0.94      0.94      0.94       114
```

[37]:

```
# Save Gaussian Naive Bayes model

joblib.dump(
    nb_model,
    "model/naive_bayes.joblib"
)
```

```
svm_recall = recall_score(
    y_test, svm_pred, zero_division=0
)
svm_f1 = f1_score(
    y_test, svm_pred, zero_division=0
)
svm_mcc = matthews_corrcoef(y_test, svm_pred)

print("SVM - EVALUATION RESULTS")
print("-" * 40)

print(f"Accuracy : {svm_accuracy:.4f}")
print(f"AUC      : {svm_auc:.4f}")
print(f"Precision: {svm_precision:.4f}")
print(f"Recall   : {svm_recall:.4f}")
print(f"F1 Score : {svm_f1:.4f}")
print(f"MCC      : {svm_mcc:.4f}")
```

```
SVM - EVALUATION RESULTS
----------------------------------------
Accuracy : 0.9737
AUC      : 0.9944
Precision: 0.9756
Recall   : 0.9524
F1 Score : 0.9639
MCC      : 0.9433
```

[43]:

```
# SVM - Confusion Matrix

svm_cm = confusion_matrix(
    y_test,
    svm_pred,
    labels=[0, 1]
)

print("SVM - CONFUSION MATRIX")
print(svm_cm)

print("\nCLASSIFICATION REPORT")
print(classification_report(
    y_test,
    svm_pred,
    labels=[0, 1],
    target_names=["Benign", "Malignant"],
    zero_division=0
))
```

```
SVM - CONFUSION MATRIX
[[71  1]
 [ 2 40]]

CLASSIFICATION REPORT
              precision    recall  f1-score   support

      Benign       0.97      0.99      0.98        72
   Malignant       0.98      0.95      0.96        42

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114
```

[44]:

```
# Save SVM model

joblib.dump(
    svm_model,
    "model/svm.joblib"
)

display(pd.DataFrame(model_results).round(4))
```

```
SVM results stored successfully!
```

	ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
0	Logistic Regression	0.9649	0.9960	0.9750	0.9286	0.9512	0.9245
1	Decision Tree	0.9298	0.9246	0.9048	0.9048	0.9048	0.8492
2	KNN	0.9561	0.9823	0.9744	0.9048	0.9383	0.9058
3	Naive Bayes	0.9386	0.9934	1.0000	0.8333	0.9091	0.8715
4	Support Vector Machine	0.9737	0.9944	0.9756	0.9524	0.9639	0.9433

```
BEST MODEL FOR EACH METRIC
----------------------------------------------
Accuracy: Support Vector Machine (0.9737)
AUC: Logistic Regression (0.9960)
Precision: Naive Bayes (1.0000)
Recall: Support Vector Machine (0.9524)
F1: Support Vector Machine (0.9639)
MCC: Support Vector Machine (0.9433)

BEST OVERALL MODEL (BASED ON F1 SCORE)
Model: Support Vector Machine
F1 Score: 0.9639
```

[48]:

```
# Phase 3 - Step 1: Prepare Streamlit project folder

import os
import shutil

project_folder = "breast_cancer_streamlit"

os.makedirs(project_folder, exist_ok=True)
os.makedirs(f"{project_folder}/model", exist_ok=True)

# Copy the five trained models
model_files = [
    "logistic_regression.joblib",
    "decision_tree.joblib",
    "knn.joblib",
    "naive_bayes.joblib",
    "svm.joblib"
]

for filename in model_files:
    shutil.copy(
        f"model/{filename}",
        f"{project_folder}/model/{filename}"
    )

# Copy the test dataset
```

Lab - lab - 57445 - 57445 - Google Chrome

g2.prayogshala.bits-pilani.ac.in/web-rdp/#/client/ODI1MjcAYwBwb3N0Z3Jlc3Fs?token=2AA15842E5A189DFCB148401FDEC9FC0EF73BB7F16469DE40FA2B414A32E9676

localhost:8947/edit/workspace/ML_assignment_2.ipynb?

jupyter ML_assignment_2 Last Checkpoint: 1 hour ago

File Edit View Settings Help

Trusted

Code

JupyterLab Python [conda env:base]

```
OK: model/naive_bayes.joblib
OK: model/svm.joblib
```

[48]:

```
# Phase 3 - Step 1: Prepare Streamlit project folder

import os
import shutil

project_folder = "breast_cancer_streamlit"

os.makedirs(project_folder, exist_ok=True)
os.makedirs(f"{project_folder}/model", exist_ok=True)

# Copy the five trained models
model_files = [
    "logistic_regression.joblib",
    "decision_tree.joblib",
    "knn.joblib",
    "naive_bayes.joblib",
    "svm.joblib"
]

for filename in model_files:
    shutil.copy(
        f"model/{filename}",
        f"{project_folder}/model/{filename}"
    )

# Copy the test dataset
shutil.copy(
    "test_data.csv",
    f"{project_folder}/test_data.csv"
)

print("Streamlit project folder created successfully!")

print("\nProject files:")
for root, dirs, files in os.walk(project_folder):
    for file in files:
        print(os.path.join(root, file))
```

```
Streamlit project folder created successfully!

Project files:
breast_cancer_streamlit/app.py
breast_cancer_streamlit/requirements.txt
breast_cancer_streamlit/test_data.csv
breast_cancer_streamlit/model/svm.joblib
breast_cancer_streamlit/model/naive_bayes.joblib
breast_cancer_streamlit/model/logistic_regression.joblib
breast_cancer_streamlit/model/decision_tree.joblib
breast_cancer_streamlit/model/knn.joblib
```

[49]:

```
%%writefile breast_cancer_streamlit/app.py

from pathlib import Path
import joblib
import pandas as pd
import streamlit as st

from sklearn.metrics import (
    accuracy_score,
    roc_auc_score,
    precision_score,
    recall_score,
    f1_score,
    matthews_corrcoef,
    confusion_matrix,
    classification_report
)
```

26°C Mostly cloudy

Search

ENG IN

08:30 AM 07-11-2026

Lab - lab - 57445 - 57445 - Google Chrome

g2.prayogshala.bits-pilani.ac.in/web-rdp/#/client/ODI1MjcAYwBwb3N0Z3Jlc3Fs?token=2AA15842E5A189DFCB148401FDEC9FC0EF73B87F16469DE40FA2B414A32E9676

localhost:8947/edit/workspace/ML_assignment_2.ipynb?factory=Panel-preview 67%

jupyter ML_assignment_2 Last Checkpoint: 1 hour ago

File Edit View Settings Help Trusted

Code JupyterLab Python [conda env:base] *

```
breast_cancer_streamlit/model/knn.joblib
```

[43]:

```
%%writefile breast_cancer_streamlit/app.py

from pathlib import Path
import joblib
import pandas as pd
import streamlit as st

from sklearn.metrics import (
    accuracy_score,
    roc_auc_score,
    precision_score,
    recall_score,
    f1_score,
    matthews_corrcoef,
    confusion_matrix,
    classification_report
)

# 1. Configure application
st.set_page_config(
    page_title="Breast Cancer Classification",
    page_icon="🎗",
    layout="wide"
)

BASE_DIR = Path(__file__).resolve().parent

MODEL_FILES = {
    "Logistic Regression": "logistic_regression.joblib",
    "Decision Tree": "decision_tree.joblib",
    "KNN": "knn.joblib",
    "Naive Bayes": "naive_bayes.joblib",
    "Support Vector Machine": "svm.joblib"
}

@st.cache_resource
def load_model(path):
    return joblib.load(path)

st.title("🎗 Breast Cancer Classification")
st.write("Evaluate five pretrained ML models without retraining.")
st.caption("Educational demonstration only. Not for clinical diagnosis.")

# 2. Select model
selected_model = st.selectbox(
    "Select Machine Learning Model",
    list(MODEL_FILES.keys()),
    key="model_selector"
)

# 3. Select dataset source
data_source = st.radio(
    "Choose Test Dataset Source",
    [
        "Use Sample Test Dataset",
        "Upload CSV or Excel File"
    ],
    key="dataset_source"
)

if data_source == "Use Sample Test Dataset":
    source = BASE_DIR / "test_data.csv"

    if not source.is_file():
        st.error("test_data.csv not found in project folder.")
        st.stop()

else:
    source = st.file_uploader(
        "Upload Test Dataset",
        type=["csv", "xlsx"],
        key="test_dataset_uploader"
    )

    if source is None:
        st.info("Please upload a CSV or Excel file.")
        st.stop()

# 4. Read dataset
try:
    filename = source.name.lower()

    if filename.endswith(".csv"):
        test_df = pd.read_csv(source)
    elif filename.endswith(".xlsx"):
        test_df = pd.read_excel(source)
    else:
        st.error("Only CSV and XLSX files are supported.")
        st.stop()

except Exception as error:
    st.error(f"Unable to read dataset: {error}")
    st.stop()

if test_df.empty:
    st.error("Dataset is empty.")
    st.stop()

test_df.columns = test_df.columns.astype(str).str.strip()

if test_df.columns.duplicated().any():
    st.error("Duplicate column names found.")
    st.stop()

st.subheader("Dataset Preview")
st.dataframe(test_df.head(10), use_container_width=True)
st.write("Total Samples:", len(test_df))

# 5. Prepare target labels
if "target" in test_df.columns:
    labels = pd.to_numeric(
        test_df["target"],
        errors="coerce"
    )

elif "Diagnosis" in test_df.columns:
    labels = (
        test_df["Diagnosis"]
        .astype(str)
        .str.strip()
        .str.upper()
        .map({"M": 1, "B": 0})
    )

else:
    st.error("Dataset must contain target or Diagnosis column.")
    st.stop()

if labels.isna().any() or not labels.isin([0, 1]).all():
    st.error("Target labels must be 0/1 or Diagnosis M/B.")
    st.stop()

y_test_app = labels.astype(int)

X_test_app = test_df.drop(
    columns=["target", "Diagnosis", "ID", "id"],
    errors="ignore"
)

# 6. Load saved model
model_path = BASE_DIR / "model" / MODEL_FILES[selected_model]

if not model_path.is_file():
    st.error(f"Model file missing: {model_path.name}")
    st.stop()

try:
    loaded_model = load_model(str(model_path))
except Exception as error:
    st.error(f"Model loading failed: {error}")
    st.stop()

# 7. Validate input features
if hasattr(loaded_model, "feature_names_in_"):
    expected_features = list(
        loaded_model.feature_names_in_
    )

    missing = [
        col for col in expected_features
        if col not in X_test_app.columns
    ]

    extra = [
        col for col in X_test_app.columns
        if col not in expected_features
    ]

    if missing or extra:
        st.error(
            f"Feature mismatch. Missing: {missing}; "
            f"Unexpected: {extra}"
        )
        st.stop()

    X_test_app = X_test_app[expected_features]

elif X_test_app.shape[1] != 30:
    st.error("Expected 30 input features.")
    st.stop()

X_test_app = X_test_app.apply(
    pd.to_numeric,
    errors="coerce"
)

if X_test_app.isna().any().any():
    st.error("Feature data contains missing or invalid values.")
    st.stop()

# 8. Predict using pretrained model
try:
    predictions = loaded_model.predict(X_test_app)

    class_probabilities = loaded_model.predict_proba(
        X_test_app
    )

    positive_index = list(
        loaded_model.classes_
    ).index(1)

    probabilities = class_probabilities[:, positive_index]

except Exception as error:
    st.error(f"Prediction failed: {error}")
    st.stop()

st.success(f"{selected_model} loaded successfully!")

# 9. Display predictions
prediction_df = pd.DataFrame({
    "Actual": y_test_app.to_numpy(),
    "Predicted": predictions,
    "Malignant Probability": probabilities
})

prediction_df["Result"] = (
```

Lab - lab - 57445 - 57445 - Google Chrome
g2.prayogshala.bits-pilani.ac.in/web-rdp/#/client/ODI1MjcAYwBwb3N0Z3Jlc3Fs?token=2AA15842E5A189DFCB148401FDEC9FC0EF73BB7F16469DE40FA2B414A32E9676
localhost:8947/edit/workspace/ML_assignment_2.ipynb?factory=Panel-preview
67%


```
prediction_df = pd.DataFrame({
    "Actual": y_test_app.to_numpy(),
    "Predicted": predictions,
    "Malignant Probability": probabilities
})

prediction_df["Result"] = [
    "Correct" if actual == predicted else "Incorrect"
    for actual, predicted in zip(y_test_app, predictions)
]

st.subheader("Prediction Results")
st.dataframe(prediction_df, use_container_width=True)

# 10. Calculate six metrics
accuracy = accuracy_score(y_test_app, predictions)

if y_test_app.nunique() == 2:
    auc = roc_auc_score(y_test_app, probabilities)
else:
    auc = float("nan")

precision = precision_score(
    y_test_app, predictions, zero_division=0
)
                                    ...
)

cm_df = pd.DataFrame(
    cm,
    index=["Actual Benign", "Actual Malignant"],
    columns=["Predicted Benign", "Predicted Malignant"]
)

st.dataframe(cm_df, use_container_width=True)

# ---------------------------------------------
# 12. Classification Report Table
# ---------------------------------------------

st.subheader("Classification Report")

# Get classification report as a dictionary
report_dict = classification_report(
    y_test_app,
    predictions,
    labels=[0, 1],
    target_names=["Benign", "Malignant"],
    output_dict=True,
    zero_division=0
)

# Convert report into a DataFrame
report_df = pd.DataFrame(report_dict).transpose()

# Remove accuracy row because it has a
# different structure from class-level metrics
report_df = report_df.drop(
    index="accuracy",
    errors="ignore"
)

# Rename rows
report_df = report_df.rename(
    index={
        "macro avg": "Macro Average",
        "weighted avg": "Weighted Average"
    }
)

# Rename columns
report_df = report_df.rename(
    columns={
        "precision": "Precision",
        "recall": "Recall",
        "f1-score": "F1 Score",
        "support": "Support"
    }
)

# Show metrics in a clean table
st.dataframe(
    report_df.style
        .format({
            "Precision": "{:.2f}",
            "Recall": "{:.2f}",
            "F1 Score": "{:.2f}",
            "Support": "{:.0f}"
        })
        .background_gradient(
            cmap="Blues",
            subset=["Precision", "Recall", "F1 Score"]
        ),
    use_container_width=True,
    hide_index=False
)

# Show overall accuracy separately
st.metric(
    label="Overall Accuracy",
    value=f"{accuracy_score(y_test_app, predictions) * 100:.2f}%"
)


# 13. Download predictions
st.download_button(
    "Download Predictions as CSV",
    data=prediction_df.to_csv(index=False),
    file_name="prediction_results.csv",
    mime="text/csv",
    key="download_predictions"
)
```

```
Overwriting breast_cancer_streamlit/app.py
```

[50]:
```
%%writefile breast_cancer_streamlit/requirements.txt
streamlit==1.66.1
pandas==2.2.3
numpy==2.4.6
scikit-learn==1.8.1
joblib==1.6.0
openpyxl==3.1.5
```

```
Overwriting breast_cancer_streamlit/requirements.txt
```

[51]:
```
%pip install -r
```

```
Usage:
  /home/cloud/anaconda3/bin/python -m pip install [options] <requirement specifier> [package-index-options] ...
  /home/cloud/anaconda3/bin/python -m pip install [options] -r <requirements file> [package-index-options] ...
  /home/cloud/anaconda3/bin/python -m pip install [options] [-e] <vcs project url> ...
  /home/cloud/anaconda3/bin/python -m pip install [options] [-e] <local project path> ...
  /home/cloud/anaconda3/bin/python -m pip install [options] <archive url/path> ...

-r option requires 1 argument
Note: you may need to restart the kernel to use updated packages.
```

[52]:
```
# Phase 3 - Step 4: Verify project files

import os

project_folder = "breast_cancer_streamlit"

required_files = [
    "app.py",
    "requirements.txt",
    "test_data.csv",
    "model/logistic_regression.joblib",
    "model/decision_tree.joblib",
    "model/knn.joblib",
    "model/naive_bayes.joblib",
    "model/svm.joblib"
]

print("PROJECT FILE VERIFICATION")
print("-" * 45)

all_present = True

for filename in required_files:
    filepath = os.path.join(project_folder, filename)

    if os.path.isfile(filepath):
        print(f"OK: {filename}")
    else:
        print(f"MISSING: {filename}")
```

Lab - lab - 57445 - 57445 - Google Chrome

g2.prayogshala.bits-pilani.ac.in/web-rdp/#/client/ODI1MjcAYwBwb3N0Z3Jlc3Fs?token=2AA15842E5A189DFCB148401FDEC9FC0EF73B87F16469DE40FA2B414A32E9676

localhost:8947/edit/workspace/ML_assignment_2.ipynb?factory=Panel-preview 67%

jupyter ML_assignment_2 Last Checkpoint: 1 hour ago

File Edit View Settings Help Trusted

Code JupyterLab Python [conda env:base] *

[52]:
```
# Phase 3 - Step 4: Verify project files

import os

project_folder = "breast_cancer_streamlit"

required_files = [
    "app.py",
    "requirements.txt",
    "test_data.csv",
    "model/logistic_regression.joblib",
    "model/decision_tree.joblib",
    "model/knn.joblib",
    "model/naive_bayes.joblib",
    "model/svm.joblib"
]

print("PROJECT FILE VERIFICATION")
print("-" * 45)

all_present = True

for filename in required_files:
    filepath = os.path.join(project_folder, filename)

    if os.path.isfile(filepath):
        print(f"OK: {filename}")
    else:
        print(f"MISSING: {filename}")
        all_present = False

print("-" * 45)

if all_present:
    print("All required project files are present!")
else:
    print("Some project files are missing.")
```

```
PROJECT FILE VERIFICATION
---------------------------------------------
OK: app.py
OK: requirements.txt
OK: test_data.csv
OK: model/logistic_regression.joblib
OK: model/decision_tree.joblib
OK: model/knn.joblib
OK: model/naive_bayes.joblib
OK: model/svm.joblib
---------------------------------------------
All required project files are present!
```

[53]:
```
# Phase 3 - Step 5: Test saved models

import joblib
import pandas as pd

project_folder = "breast_cancer_streamlit"

test_df = pd.read_csv(
    f"{project_folder}/test_data.csv"
)

X_app_test = test_df.drop(columns=["target"])

model_files = {
    "Logistic Regression": "logistic_regression.joblib",
    "Decision Tree": "decision_tree.joblib",
    "KNN": "knn.joblib",
    "Naive Bayes": "naive_bayes.joblib",
    "Support Vector Machine": "svm.joblib"
}

print("SAVED MODEL TEST")
print("-" * 45)

for model_name, filename in model_files.items():

    model_path = f"{project_folder}/model/{filename}"

    loaded_model = joblib.load(model_path)

    predictions = loaded_model.predict(X_app_test)

    print(
        f"{model_name}: "
        f"Successfully predicted {len(predictions)} samples"
    )

print("\nAll saved models tested successfully!")
```

```
SAVED MODEL TEST
---------------------------------------------
Logistic Regression: Successfully predicted 114 samples
Decision Tree: Successfully predicted 114 samples
KNN: Successfully predicted 114 samples
Naive Bayes: Successfully predicted 114 samples
Support Vector Machine: Successfully predicted 114 samples

All saved models tested successfully!
```

[54]:
```
%pip install streamlit
```

```
Requirement already satisfied: streamlit in /home/cloud/anaconda3/lib/python3.12/site-packages (1.67.0)
Requirement already satisfied: altair<5.4.0,!=5.4.1,<7,>=5.0.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (6.3.0)
Requirement already satisfied: click<9,>=7.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (8.1.7)
Requirement already satisfied: numpy<3,>=1.23.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (2.3.4)
Requirement already satisfied: packaging>=20 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (24.1)
Requirement already satisfied: pandas<4,>=1.3.3 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (2.2.3)
Requirement already satisfied: pillow<14,>=9.2.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (11.0.0)
Requirement already satisfied: pydeck<1,>=0.8.0b4 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (0.9.1)
Requirement already satisfied: protobuf<8,>=3.20.1 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (7.36.2)
Requirement already satisfied: pyarrow<25.0.0,>=7.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (26.0.1)
Requirement already satisfied: requests<3,>=2.27 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (2.32.3)
Requirement already satisfied: typing-extensions<5,>=4.10.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (4.16.0)
Requirement already satisfied: tzlocal<6,>=1.88.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (1.10.0)
Requirement already satisfied: uvicorn<2,>=0.30.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (0.34.0)
Requirement already satisfied: anyio<5,>=4.0.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (4.6.2)
Requirement already satisfied: python-multipart<1,>=0.0.10 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (0.0.33)
Requirement already satisfied: websockets<16,>=12.0.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (17.4)
Requirement already satisfied: itsdangerous<3,>=2.1.2 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (2.2.0)
Requirement already satisfied: watchdog<8,>=2.1.5 in /home/cloud/anaconda3/lib/python3.12/site-packages (from streamlit) (4.0.1)
Requirement already satisfied: jinja2 in /home/cloud/anaconda3/lib/python3.12/site-packages (from altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (3.1.4)
Requirement already satisfied: jsonschema>=3.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (4.23.0)
Requirement already satisfied: narwhals>=2.0.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (2.20.0)
Requirement already satisfied: idna>=2.8 in /home/cloud/anaconda3/lib/python3.12/site-packages (from anyio<5,>=4.0.0->streamlit) (3.7)
Requirement already satisfied: sniffio>=1.1 in /home/cloud/anaconda3/lib/python3.12/site-packages (from anyio<5,>=4.0.0->streamlit) (1.3.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /home/cloud/anaconda3/lib/python3.12/site-packages (from pandas<4,>=1.3.3->streamlit) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /home/cloud/anaconda3/lib/python3.12/site-packages (from pandas<4,>=1.3.3->streamlit) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /home/cloud/anaconda3/lib/python3.12/site-packages (from pandas<4,>=1.3.3->streamlit) (2023.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /home/cloud/anaconda3/lib/python3.12/site-packages (from requests<3,>=2.27->streamlit) (3.3.2)
Requirement already satisfied: urllib3<3,>=1.21.1 in /home/cloud/anaconda3/lib/python3.12/site-packages (from requests<3,>=2.27->streamlit) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /home/cloud/anaconda3/lib/python3.12/site-packages (from requests<3,>=2.27->streamlit) (2024.12.14)
Requirement already satisfied: h11>=0.8 in /home/cloud/anaconda3/lib/python3.12/site-packages (from uvicorn<2,>=0.30.0->streamlit) (0.14.0)
Requirement already satisfied: MarkupSafe>=2.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from jinja2->altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (2.1.3)
Requirement already satisfied: attrs>=22.2.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from jsonschema>=3.0->altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (23.1.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /home/cloud/anaconda3/lib/python3.12/site-packages (from jsonschema>=3.0->altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (2023.7.1)
Requirement already satisfied: referencing>=0.28.4 in /home/cloud/anaconda3/lib/python3.12/site-packages (from jsonschema>=3.0->altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (0.30.2)
Requirement already satisfied: rpds-py>=0.7.1 in /home/cloud/anaconda3/lib/python3.12/site-packages (from jsonschema>=3.0->altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (0.10.6)
Requirement already satisfied: six>=1.5 in /home/cloud/anaconda3/lib/python3.12/site-packages (from python-dateutil>=2.8.2->pandas<4,>=1.3.3->streamlit) (1.16.0)
Note: you may need to restart the kernel to use updated packages.
```

[55]:
```
# Check library versions for deployment compatibility

import importlib.metadata as metadata

packages = [
    "streamlit",
    "pandas",
    "numpy",
    "scikit-learn",
    "joblib",
    "openpyxl"
]

for package in packages:
    try:
        print(f"{package}: {metadata.version(package)}")
    except metadata.PackageNotFoundError:
        print(f"{package}: Not installed")
```

```
streamlit: 1.67.0
pandas: 2.2.3
numpy: 2.3.4
scikit-learn: 1.0.1
joblib: 1.4.2
openpyxl: Not installed
```

[56]:
```
import subprocess
import sys
import os
import time

PORT = 8501
APP_PATH = "breast_cancer_streamlit/app.py"

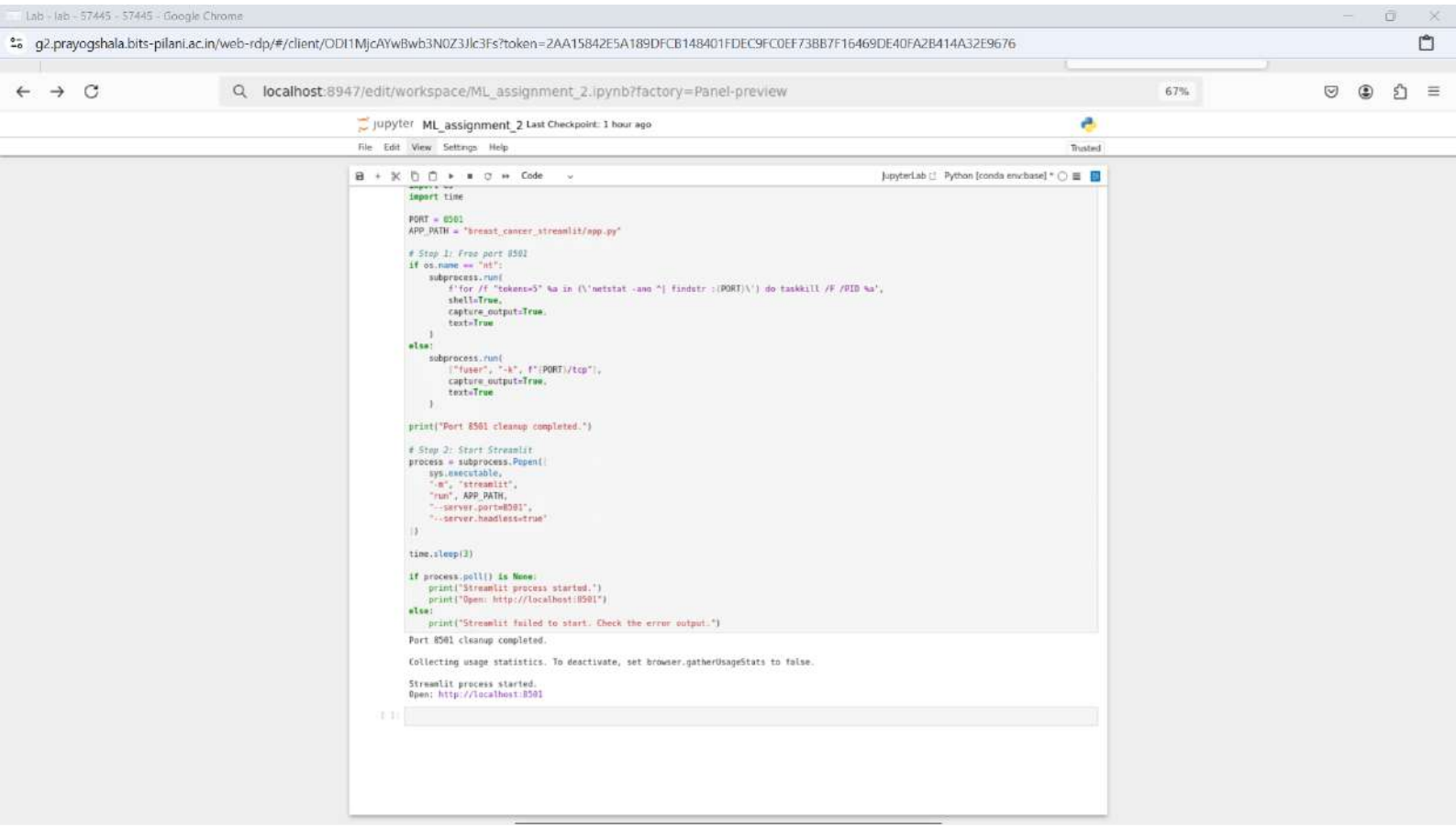
# Step 1: Free port 8501
if os.name == "nt":
    subprocess.run(
        f'for /f "tokens=5" %a in (\'netstat -ano ^| findstr :{PORT}\') do taskkill /F /PID %a',
        shell=True,
        capture_output=True,
        text=True
    )
else:
    subprocess.run(
        ["fuser", "-k", f"{PORT}/tcp"],
        capture_output=True,
        text=True
    )

print("Port 8501 cleanup completed.")

# Step 2: Start Streamlit
process = subprocess.Popen([
    sys.executable,
    "-m", "streamlit",
    "run", APP_PATH,
    "--server.port=8501",
    "--server.headless=true"
])

time.sleep(3)

if process.poll() is None:
    print("Streamlit process started.")
    print("Open: http://localhost:8501")
else:
```


Lab - lab - 57445 - 57445 - Google Chrome
g2.prayogshala.bits-pilani.ac.in/web-rdp/#/client/ODI1MjcAYwBwb3N0Z3Jlc3Fs?token=2AA15842E5A189DFCB148401FDEC9FC0EF73BB7F16469DE40FA2B414A32E9676
localhost:8947/edit/workspace/ML_assignment_2.ipynb?factory=Panel-preview
67%
jupyter ML_assignment_2 Last Checkpoint: 1 hour ago
File Edit View Settings Help
Trusted
Code
JupyterLab
Python [conda env:base]
import time
PORT = 8501
APP_PATH = "breast_cancer_streamlit/app.py"
Step 1: Free port 8501
if os.name == "nt":
subprocess.run(
f'for /f "tokens=5" %a in (\'netstat -ano ^| findstr :{PORT}\') do taskkill /F /PID %a',
shell=True,
capture_output=True,
text=True
)
else:
subprocess.run(
["fuser", "-k", f"{PORT}/tcp"],
capture_output=True,
text=True
)
print("Port 8501 cleanup completed.")
Step 2: Start Streamlit
process = subprocess.Popen([
sys.executable,
"-m", "streamlit",
"run", APP_PATH,
"--server.port=8501",
"--server.headless=true"
])
time.sleep(3)
if process.poll() is None:
print("Streamlit process started.")
print("Open: http://localhost:8501")
else:
print("Streamlit failed to start. Check the error output.")
Port 8501 cleanup completed.
Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.
Streamlit process started.
Open: http://localhost:8501